Linear Algebra and Applications Graded Assignment 3 - Page Rank

Deadline: Nov 28 2026 at noon

In this assignment you are not allowed to use the module `Networkx`.

This document is associated with the files `pagerank.py` and the data files: `web_stanford.txt`, `psh-uefa-2017-2018.csv`, which can be obtained by pulling your git repository. The file `pagerank.py` is the only one that needs to be edited and submitted.

Task 1

Subtask 1.a

Write a class for representing directed graphs via their adjacency matrices. (Be careful about the notation: the standard representation of an adjacency matrix for directed graphs has "from" nodes on rows and "to" nodes on columns. However, in this assignment as well as in the corresponding slides we use the convention of having "from" nodes on columns and "to" nodes on rows.)

The constructor of the class should accept an $n \times n$ adjacency matrix A and a list of node labels in string form (such as `['a', 'b', 'c', 'd']`) defaulting to `None`. The constructor should then modify A into $\widetilde{A}$ as shown in the slides so that there are no sinks in the corresponding graph, and finally calculate the $\widehat{A}$ matrix:

$$\widehat{A}_{ij} = \frac{\widetilde{A}_{ij}}{\sum_{k=1} \widetilde{A}_{kj}}.$$

Save $\widehat{A}$ and the list of labels as attributes of the `DiGraph` object. Use the integer numbers $[0, 1, \ldots, n-1]$ as the labels only if none are provided. Finally, raise a `ValueError` if the number of labels is not equal to the number of nodes in the graph. (Hint: use array broadcasting to compute $\widehat{A}$ efficiently. The Numpy function `where` could help you.)

In the docstring example you find an example graph with the expected output.

Subtask 1.b

Add the following methods to your class from the previous Subtask. Each should accept a damping factor ϵ (defaulting to 0.85), compute the PageRank vector $\mathbf{x}$, and return a dictionary mapping label i to its PageRank value x_i.

1. `linsolve()`: solve for $\mathbf{x}$ (Method I from slides) after substituting the fact that $\mathbf{1}\mathbf{1}^\mathsf{T}\mathbf{x} = \mathbf{1}$ since $\sum_i x_t(i) = 1$:

$$\begin{aligned}\left(I - \epsilon\widehat{A} - (1-\epsilon)\tfrac{1}{n}\mathbf{1}\mathbf{1}^\mathsf{T}\right)\mathbf{x} &= \mathbf{0} \\ \left(I - \epsilon\widehat{A}\right)\mathbf{x} &= \tfrac{1-\epsilon}{n}\mathbf{1}\end{aligned}$$

2. `eigensolve()`: solve $\bar{A}\mathbf{x} = \mathbf{x}$ (Method II from slides) for $\mathbf{x}$.

3. `itersolve()`: in addition to ϵ, accept an integer `maxiter` and a float `tol`. Iterate until

$$||\mathbf{x}_t - \mathbf{x}_{t-1}||_2 < \texttt{tol} \quad \text{or} \quad t > \texttt{maxiter}.$$

   Use $\mathbf{x}_0 = [\frac{1}{n}, \frac{1}{n}, \ldots, \frac{1}{n}]^\mathsf{T}$ as the initial vector (any positive vector that sums to 1 will do, but this assumes equal starting probabilities). Use Euclidean scaling before and throughout the iterations.

In the docstring example you can test that the three methods yield the same results. For the graph of the lecture with $\epsilon = 0.85$, you should get the following dictionary mapping labels to PageRank values.

```
{'a': 0.095758635, 'b': 0.274158285,
 'c': 0.355924792, 'd': 0.274158285}
```

Make sure your methods return probability distributions. If needed normalize the resulting eigenvector so that its entries sum to 1. Note also that any nonzero scalar multiple of an eigenvector is an eigenvector. Thus also vectors of the form $\vec{v} = -n\vec{w}$, $n > 0$ for an eigenvector w are eigenvectors.

Subtask 1.c

Write a function that accepts a dictionary mapping labels to PageRank values, like the outputs in the previous Subtask, and returns a list of labels sorted **from highest to lowest** value. (Hint: look for the built-in function `sorted()`.)

For the graph in the slides with $\epsilon = 0.85$, the list is `['c', 'b', 'd', 'a']` (or `['c', 'd', 'b', 'a']`, since `b` and `d` have the same PageRank value).

Round the steady state probabilities to the 8th decimal and break ties by lexicographic order (for example, the ranking for the graph in the slides with $\epsilon = 0.85$ must be `['c', 'b', 'd', 'a']` while `['c', 'd', 'b', 'a']` becomes an incorrect order).

Looking at the graph visually, it is easy to see why `a` has the lowest PageRank value: the only other node that points to it is `b`. It also makes sense that `c` has the highest ranking, since `c` and `d` both have edges from the other three nodes pointing to them, but `d` only has one edge (pointing to `c`), while `c` points to both `b` and `d`. In other words, at each step `d` distributes all of its importance to `c`, while `c` splits its importance between `b` and `d`.

Of course, constructing rankings is much more difficult to do by hand when there are more than just a few nodes in the graph.

Task 2

The file `web_stanford.txt` contains a subset of the information on [Stanford University webpages](`http://snap.stanford.edu/data/web-Stanford.html`) and the hyperlinks between them, gathered in 2002. Each line of the file is formatted as `a/b/c/d/e/f...`, meaning the webpage with ID `a` has hyperlinks to webpages with IDs `b`, `c`, `d`, and so on. In fact the IDs are integer numbers but for the sake of this assignment, handle them as strings.

Write a function that accepts a damping factor ϵ defaulting to 0.85. Read the data and get a list of the n unique page IDs in the file (the labels). Construct the $n \times n$ adjacency matrix of the graph where node j points to

node i if webpage j has a hyperlink to webpage i. Use your class from Subtask 1.a and its `itersolve()` method from Subtask 1.b to compute the PageRank values of the webpages, then rank them with your function from Subtask 1.c. Return the ranked list of webpage IDs. (Hint: after constructing the list of webpage IDs, make a dictionary that maps a webpage ID to its index in the list. The values are the row/column indices in the adjacency matrix for each label.)

With $\epsilon = 0.85$, the top three ranked webpage IDs are `'98595'`, `'32791'`, and `'28392'`. Note, break ties lexicographically, e.g., if `'1123'` and `'345'` have the same score then rank `'1123'` before `'345'`.

Task 3

The file `psh-uefa-2018-2019.csv` contains data for men's football teams in Europe for the season 2018-2019 from the main leagues in Italy, England, France, Scotland, Spain, Germany, Greece, Belgium, Holland, Portugal, Turkey, together with Champions League and Europa League. Each line represents a different game, formatted `home_team,away_team,home_goals,away_goals`. Write a function that accepts a filename and a damping factor ϵ defaulting to 0.85. Read the specified file and get a list of the n unique teams in the file. Construct the $n \times n$ adjacency matrix of the graph where node j points to node i with weight w if team j was defeated by team i in w games. That is, **edges point from losers to winners**. Ignore draw games. Use your class from Subtask 1.a and its 'itersolve()' method from Subtask 1.b to compute the PageRank values of the teams, then rank them with your function from Subtask 1.c. Return the ranked list of team names.

Using `psh-uefa-2018-2019.csv` with $\epsilon = 0.85$, the top three ranked teams in the current season should be Liverpool, Athletic Madrid, Paris SG.

The damping factor ϵ acts as an "upset" factor: a larger ϵ puts more emphasis on win history; a smaller ϵ allows more randomness in the system, giving underdog teams a higher probability of defeating a team with a better record.

Note that we did not take into account weights derived from home-away situations, goal difference, importance of the match and aggregate results from Europa and Champions league.

It is also worth noting that the sink-fixing procedure is still reasonable for this model because it gives every other team 'equal' likelihood of beating

an undefeated team. That is, the additional arcs don't provide an extra advantage to any one team.

If you are interested in comparing this way of ranking against the FIFA international ranking for national teams, you can run your procedures on the data set avaialble from [kaggle] (`https://www.kaggle.com/martj42/international-football-results-from-1872-to-2017`), which contains the results of international football matches of national teams from 1872 to 2021.